

DOCUMENTO DE DISEÑO DE SOFTWARE

Lydia Agent V2

System Design Document (SDD)

Versión: 0.17.0

Fecha: 01/07/2026

Proyecto: Jobs/Arquant/lydia-agent

Proveedor de modelo: Rafw007/Qwen3.6-A3B-CLaude Coder (custom)

Repositorio: Jobs/Arquant/lydia-agent

1. ¿Qué es Lydia Agent V2?

Lydia Agent V2 es un agente de IA personal de código abierto construido por **Alexnader Villanueva**. Ejecuta el mismo core de agente a través de múltiples interfaces: CLI, gateway de mensajería, TUI terminal y aplicación de escritorio Electron. Aprende entre sesiones (memoria + skills), delega a subagentes, ejecuta tareas programadas y controla un terminal y navegador reales. Se extiende principalmente mediante **plugins y skills**, no haciendo crecer el core.

Líneas de código analizadas: más de **50,000 líneas Python** en el árbol de código, distribuidas en **141 módulos** del agent core, **195 módulos CLI**, **108 herramientas**, **67 archivos gateway** y **175 plugins**. La versión actual es **0.17.0**.

1.1 Propiedades de diseño fundamentales

Propiedad	Impacto
Caché de prompt por conversación	Cada conversación larga reutiliza un prefijo en caché. Mutar el contexto pasado, cambiar toolsets o reconstruir el system prompt invalida el caché y multiplica el costo del usuario.
Core como narrow waist	Cada core tool se envía en cada llamada API. La barrera para un nuevo core tool es altísima. La mayoría de nueva capacidad debe llegar como CLI command + skill, plugin o MCP.
Config vs. Secrets	<code>.env</code> solo para credenciales (API keys, tokens, passwords). Toda configuración de comportamiento (timeouts, thresholds, feature flags) va en <code>config.yaml</code> .

1.2 Componentes principales

Componente	Responsabilidad	Tamaño
Agent Core (agent/)	Loop de conversación, llamadas de modelo, ejecución de herramientas, gestión de sesión	141 archivos Python, ~7 MB
CLI (lydia_cli/)	Subcomandos `lydia`, configuración, setup wizard, dashboards, modelos	195 archivos Python, ~12 MB
Gateway (gateway/)	Daemon de mensajería multi-plataforma: Telegram, Discord, Slack, WhatsApp, Signal, Webhook, etc.	67 archivos Python, ~5.3 MB, Aun en desarrollo
Run Agent (run_agent.py)	Runner principal del agente: loop de conversación, tool calling, recovery, contexto	5,709 líneas
Cron (cron/)	Programador de tareas con entrega multi-plataforma, sugerencias catalog, jobs DB	9 archivos Python, ~485 KB, Aun en desarrollo
Tools (tools/)	60+ herramientas de línea base: terminal, browser, delegation, file, cronjob, skills, memory, etc.	108 archivos Python, ~7.3 MB, Aun en desarrollo

Plugins (plugins/)	Extensibilidad: browser, context_engine, cron_providers, dashboard_auth, memory, model-providers, platforms, security-guidance, team_pipeline, web, etc.	175 archivos Python, ~10.8 MB, Aun en desarrollo
Skills (optional-skills/)	Skills de dominio: mlops, creative, software-development, research, devops, blockchain, communication, productivity, security, health, etc.	57 archivos Python, ~5.9 MB, Parte aun en desarrollo
Browser (acp_adapter/)	Adaptador ACP (Agent Control Protocol) para control de navegador remoto	11 archivos Python, ~215 KB, Aun en desarrollo

2. Arquitectura General del Agente

El sistema sigue un patrón de **waist design**: un core de agente estrecho y reutilizable que se conecta a múltiples interfaces (CLI, TUI, gateway) y se extiende radialmente a través de plugins y skills. El core nunca debe hacer crecer su superficie: cada tool core se envía en cada llamada API, por lo que cada nueva herramienta tiene un costo permanente multiplicado por el uso.

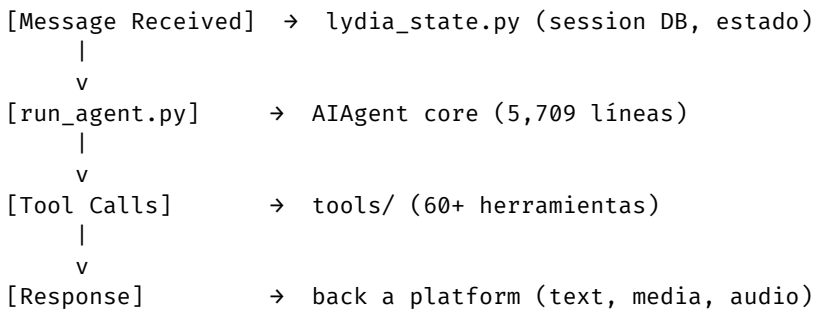
2.1 Flujo de ejecución principal (CLI/TUI)

La entrada del usuario viaja por **run_agent.py** (5,709 líneas) → agent/server.py → agent/chat_completion_helpers.py → modelo → herramienta o respuesta. Cuando el agente necesita ejecutar una acción (terminal, file, browser, delegate), el core genera un tool call que el runtime ejecuta de forma segura y devuelve el resultado al mismo mensaje de la conversación.

Fase	Componente	Descripción
1. Entrada	lydia_cli/main.py (13,668 líneas) o gateway/run.py (19,114 líneas)	Lee el mensaje del usuario desde CLI, TUI o plataforma (Telegram, Discord, etc.)
2. Preparación de contexto	agent/context_manager.py	Construye el contexto: memory (FTS5), skills hub, session search, project context, cron jobs
3. Llamada al modelo	run_agent.py → model_tools.py → providers/	OpenAI-compatible API call. Soporta Nous Portal, OpenRouter, OpenAI, custom, y 6+ backends
4. Respuesta	run_agent.py (parse, tool calls) o agent/server.py (streaming)	Si hay tool calls, se ejecutan y se reintegran; si no, se entrega la respuesta al usuario
5. Ejecución de herramientas	tools/ (108 archivos, 7.3 MB)	Cada tool se ejecuta en contexto aislado con timeout, límites de output y sandboxing
6. Feedback loop	run_agent.py + lydia_state.py (5,659 líneas)	El resultado de la herramienta se envía al modelo como mensaje de sistema, loop de tool-call

2.2 Flux completo de mensajería (Gateway)





2.3 Componentes del core

Módulo	Responsabilidad	Clave
agent/server.py	Server HTTP/gateway para el core de agente	Exposición API REST, streaming SSE
agent/session.py (628 líneas)	Gestión de sesiones (crear, listar, limpiar)	CRUD de conversaciones
agent/tools.py (1,292 líneas)	Registro central de herramientas del agente	Schema + routing
agent/auth.py (80 líneas)	Autenticación y permisos	Validador de identidad
agent/edit_approval.py (287 líneas)	Aprobación antes de edits en archivos	Safety: confirma antes de modificar
agent/events.py (280 líneas)	Sistema de eventos interno	Pub/sub para el core
agent/permissions.py (169 líneas)	Permisos y RBAC	Control de acceso
agent/provenance.py (128 líneas)	Procedencia y chain of custody	Audit trail de cada acción
lydia_state.py (5,659 líneas)	Gestión de estado persistente de la sesión	Conversaciones, historial, herramientas activas, cron
lydia_constants.py (997 líneas)	Constantes compartidas, paths de HOME, perfiles	~/.lydia (nuevo)
lydia_logging.py (613 líneas)	Centricación de logging	Configuración unificada de logs
lydia_time.py (125 líneas)	Gestión de zonas horarias	Fallback a tzdata en Windows
model_tools.py (1,260 líneas)	Normalización de modelos, cost guard, model switch	Adaptador multi-proveedor

3. Arquitectura de Gateway

El Gateway es el daemon de mensajería multi-plataforma. Su módulo principal, **gateway/run.py (19,114 líneas)**, gestiona la conexión con clientes de mensajería externa, procesa mensajes entrantes, los envía al core del agente para su procesamiento y devuelve las respuestas a la plataforma apropiada.

3.1 Componentes del Gateway

Componente	Responsabilidad
gateway/run.py (19,114 líneas)	Daemon principal. Inicia el bucle de eventos, maneja la reconexión y las notificaciones de herramientas.
platforms/	Adaptadores de plataforma para diferentes servicios de mensajería (Telegram, Discord, Slack, WhatsApp, Signal, etc.). Cada plataforma tiene su propio adaptador.
config.yaml	Configuración del gateway, incluidos tokens de API, ajustes de plataforma y opciones de autenticación.

3.2 Integración con la Plataforma

Cada plataforma requiere sus propios endpoints de API y configuración de seguridad. Las configuraciones típicas incluyen tokens de API o certificados OAuth, ajustes de WebSocket, o endpoints webhook para la entrega asincrónica de mensajes.

3.3 Funcionalidades Clave del Gateway

- **Soporte Multi-plataforma:** Conecta con varios servicios de mensajería a través de adaptadores. El soporte para plataformas puede estar en desarrollo activo.
- **Procesamiento de Mensajes:** Gestiona la entrega de mensajes entrantes y salientes, las respuestas automatizadas y el enrutamiento de comandos.
- **Gestión de Sesión Persistente:** Mantiene el contexto de la conversación y el estado entre las interacciones.
- **Escalado:** El daemon centralizado permite el acceso multi-sesión a través de múltiples canales con un único motor de agente.

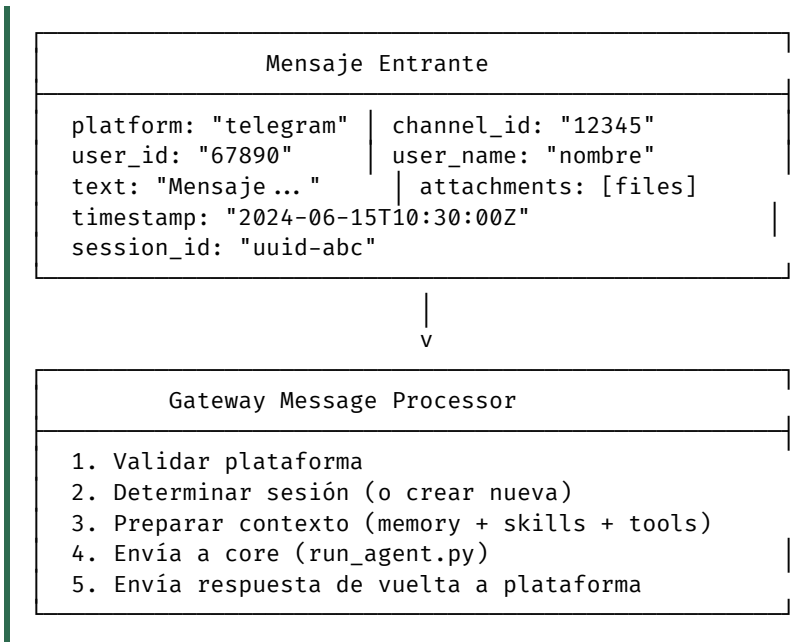
Nota: La integración con cada plataforma puede requerir ajustes específicos de API y configuraciones de seguridad. La compatibilidad con algunas plataformas puede no estar disponible completamente.

3.4 Plataformas Soportadas por el Gateway

El gateway soporta múltiples plataformas de mensajería a través de adaptadores específicos. Cada plataforma tiene su propia implementación en `gateway/platforms/` :

Plataforma	Estado	Archivo	Descripción
Telegram	Completo	telegraph/	Bot con comandos slash, inline query, multimedia
Discord	Completo	discord/	Bot con interacciones, slash commands, embeds
Slack	Completo	slack/	Bot con slash commands, Block Kit, reactions
WhatsApp	Completo	whatsapp_cloud.py	WhatsApp Cloud API oficial
Signal	Completo	signal.py	Signal Desktop + libsignal-service-daemon
Webhook	Completo	webhook.py	Endpoint genérico para cualquier servicio HTTP
MS Graph	Completo	msgraph_webhook.py	Microsoft Graph API webhook
WeiXin	Completo	weixin.py	WeChat (Weixin) Open API
BlueBubbles	Completo	bluebubbles.py	Pasarela iMessage para Mac
YuanBao	Completo	yuanbao_*.py	App móvil china (4 archivos específicos)
REST API	Completo	api_server.py	API REST propia del gateway (202 KB)

3.5 Modelo de Mensajes



4. Configuración del Sistema

4.1 Estructura del Proyecto

```
lydia-agent/                                # Directorio raíz del proyecto
├── agent/                                  # Core del agente (141 .py files, ~7 MB)
│   ├── server.py                          # Server HTTP/gateway para agente
│   ├── session.py                        # Gestión de sesiones (628 líneas)
│   ├── tools.py                          # Registro de herramientas (1,292 líneas)
│   ├── auth.py                           # Autenticación y permisos
│   ├── edit_approval.py                  # Aprobación de edits (287 líneas)
│   ├── events.py                         # Sistema de eventos (280 líneas)
│   ├── permissions.py                    # Permisos (169 líneas)
│   └── provenance.py                     # Procedencia (128 líneas)
├── lydia_cli/                             # CLI (195 .py files, ~12 MB)
│   ├── main.py                           # Punto de entrada principal (13,668 líneas)
│   ├── auth.py                           # Autenticación CLI (8,411 líneas)
│   ├── config.py                         # Configuración (7,801 líneas)
│   ├── models.py                         # Gestión de modelos (4,208 líneas)
│   ├── tools_config.py                   # Configuración de herramientas (4,258 líneas)
│   ├── gateway.py                        # Gestión del gateway (6,843 líneas)
│   ├── kanban_db.py                      # Base de datos kanban (8,723 líneas)
│   ├── kanban.py                         # Comandos kanban (2,845 líneas)
│   ├── setup.py                          # Setup wizard (3,411 líneas)
│   ├── gateway_windows.py                # Gateway en Windows (1,682 líneas)
│   ├── model_setup_flows.py              # Flujos de setup de modelos (2,827 líneas)
│   ├── web_server.py                     # Servidor web/dashboard (14,244 líneas)
│   ├── curator.py                        # Curador de memoria (619 líneas)
│   ├── plugin.py                         # Gestión de plugins (2,286 líneas)
│   ├── backup.py                         # Sistema de backup/restore (1,376 líneas)
│   ├── mcp_config.py                     # Configuración de MCP (977 líneas)
│   ├── memory_oauth.py                   # OAuth para memory providers (83 líneas)
│   ├── security_audit.py                 # Auditoría de seguridad (576 líneas)
│   └── ... (142 archivos CLI restantes)
├── run_agent.py                           # Runner del agente (5,709 líneas)
└── tools/                                 # Herramientas del agente (108 .py, ~7.3 MB)
    ├── terminal_tool.py                  # Terminal remoto (2,978 líneas)
    ├── execute_code_tool.py              # Ejecución de código (1,868 líneas)
    ├── file_tools.py                     # Operaciones de archivo (1,936 líneas)
    ├── skills_tool.py                    # Gestión de skills (1,662 líneas)
    ├── image_generation_tool.py           # Generación de imágenes (1,680 líneas)
    ├── tts_tool.py                       # Texto-a-voz (2,846 líneas)
    ├── delegation_tool.py                 # Subagentes (3,510 líneas)
    ├── mcp_tool.py                       # MCP tool gateway (4,911 líneas)
    ├── browser_tool.py                   # Navegador CamoFox (4,499 líneas)
    ├── browser_supervisor.py              # Supervisor de navegador (1,475 líneas)
    ├── cronjob_tools.py                   # Tareas cron (1,032 líneas)
    ├── memory_tool.py                     # Memory (1,146 líneas)
    ├── send_message_tool.py               # Envío multi-plataforma (1,794 líneas)
    ├── transcription_tools.py              # Transcripción (1,799 líneas)
    ├── checkpoint_manager.py              # Checkpoints (1,675 líneas)
    ├── kanban_tools.py                    # Kanban (1,669 líneas)
    ├── web_tools.py                       # Web scraping/search (1,012 líneas)
    ├── session_search_tool.py              # Búsqueda FTS5 (921 líneas)
    ├── lazy_deps.py                       # Lazy dependencies (921 líneas)
    └── approval.py                       # Approval system (2,252 líneas)
```

— process_registry.py	# Proceso / PID management (2,219 líneas)
— registry.py	# Registry de herramientas (708 líneas)
— mcp_oauth_manager.py	# MCP OAuth (720 líneas)
— skills_usage.py	# Track de skill usage (947 líneas)
— skills_hub.py	# Skills hub (4,069 líneas)
— skills_sync.py	# Sync de skills (1,182 líneas)
— voice_mode.py	# Voice mode (1,218 líneas)
— path_security.py	# Seguridad de paths (43 líneas)
— ... (78 archivos de herramientas restantes)	
— plugins/	# Sistema de plugins (175 .py, ~10.8 MB)
— model-providers/	# Model providers (28 .py, 141 items)
— platforms/	# Platform adapters (51 .py, 164 items)
— memory/	# Memory plugins (20 .py, 46 items)
— web/	# Web plugins (17 .py, 59 items)
— browser/	# Browser plugins (6 .py, 21 items)
— image_gen/	# Image gen plugins (6 .py, 30 items)
— context_engine/	# Engine de contexto (1 .py)
— cron_providers/	# Cron providers (4 .py, 6 items)
— dashboard_auth/	# Auth dashboard (4 .py, 20 items)
— disk-cleanup/	# Disk cleanup (2 .py)
— google_meet/	# Google Meet (14 .py, 19 items)
— lydia-achievements/	# Logros (2 .py, 19 items)
— kanban/	# Kanban plugin (1 .py)
— observability/	# Observabilidad (2 .py, 8 items)
— spotify/	# Spotify (3 .py, 8 items)
— teams_pipeline/	# Pipelines Teams (8 .py, 9 items)
— video_gen/	# Video gen (2 .py, 10 items)
— security-guidance/	# Security guidance (2 .py, 6 items)
— ... plugins adicionales	
— optional-skills/	# Skills de dominio (57 .py, ~5.9 MB)
— autonomous-ai-agents/	# Agentes autónomos + orquestación
— blockchain/	# Blockchain (3 .py, 12 items)
— creative/	# Contenido creativo (7 .py, 182 items)
— devops/	# DevOps (4 .py, 20 items)
— email/	# Email (2 .py, 10 items)
— finance/	# Finanzas (3 .py, 28 items)
— health/	# Salud (2 .py, 14 items)
— mcp/	# MCP (4 .py, 13 items)
— mlops/	# ML Ops (1 .py, 174 items)
— payments/	# Pagos (6 .py, 6 items)
— productivity/	# Productividad (4 .py, 29 items)
— research/	# Investigación (22 .py, 72 items)
— security/	# Seguridad (5 .py, 46 items)
— software-development/	# Dev de software (14 .py)
— web-development/	# Web dev (7 .py)
— gateway/	# Gateway (67 .py, ~5.3 MB)
— run.py	# Daemon principal (19,114 líneas)
— channels/	# Directorio y enrutamiento
— platforms/	# Adaptadores de plataforma
— telegram/	# Telegram (comandos slash, inline, multimedia)
— discord/	# Discord (interacciones, slash commands)
— slack/	# Slack (slash commands, Block Kit)
— whatsapp_cloud.py	# WhatsApp Cloud API (83 KB)
— signal.py	# Signal Desktop (69 KB)
— signal_format.py	# Formato Signal (4 KB)
— signal_rate_limit.py	# Rate limiting Signal (14 KB)
— webhook.py	# Webhook genérico (42 KB)
— msgraph_webhook.py	# MS Graph (16 KB)
— weixin.py	# WeChat (89 KB)

— bluebubbles.py	# iMessage bridge (38 KB)
— yuanbao.py	# YuanBao app china (208 KB)
— yuanbao_media.py	# YuanBao media (20 KB)
— yuanbao_proto.py	# YuanBao protocol (44 KB)
— yuanbao_sticker.py	# YuanBao stickers (20 KB)
— api_server.py	# REST API gateway (202 KB)
— _http_client_limits.py	# Límites de clientes HTTP (2 KB)
— base.py	# Base adapter (235 KB)
— helpers.py	# Helpers de plataforma (13 KB)
— agent/	# Core agente (141 .py, ~7 MB)
— server.py	# Server HTTP/gateway
— session.py	# Gestión de sesiones (628 líneas)
— tools.py	# Registro herramientas (1,292 líneas)
— edit_approval.py	# Aprobación de edits (287 líneas)
— events.py	# Sistema eventos (280 líneas)
— permissions.py	# Permisos (169 líneas)
— provenance.py	# Procedencia (128 líneas)
— cron/	# Programador cron (9 .py, ~485 KB)
— scheduler.py	# Scheduler principal
— scheduler_provider.py	# Proveedores de scheduler
— jobs.py	# Gestión de jobs
— blueprint_catalog.py	# Catálogo de blueprints
— suggestion_catalog.py	# Catálogo de sugerencias
— suggestions.py	# Generador de sugerencias
— scripts/	# Scripts de cron (2 .py)
— acp_adapter/	# Agent Control Protocol (11 .py, 215 KB)
— locales/	# Internacionalización (16 idiomas)
— en.yaml de.yaml ja.yaml ko.yaml ru.yaml	
— es.yaml zh.yaml zh-hant.yaml fr.yaml pt.yaml	
— it.yaml uk.yaml tr.yaml ga.yaml af.yaml	
— platforms/	# Plataformas de entrega
— ADDING_A_PLATFORM.md	# Guía para añadir plataforma (14 KB)
— __init__.py	# Init (1 KB)
— _http_client_limits.py	# Límites HTTP (2 KB)
— base.py	# Base (235 KB)
— api_server.py	# REST API (202 KB)
— helpers.py	# Helpers (13 KB)
— (16 archivos de plataforma)	
— docs/	# Documentación del proyecto
— website/	# Sitio web oficial (4 .py, 25 KB)
— docker/	# Dockerfiles (0 .py, 44 KB)
— nix/	# Nix flakes
— scripts/	# Scripts de utilidad (19 .py, 820 KB)
— packaging/	# Scripts de packaging (2 .py)
— tests/	# Tests (1,864 .py, 25 MB)
— assets/	# Assets estáticos (12 KB)
— .github/	# GitHub config (958 KB)
— .lydia/	# lydia config
— .logs/	# Logs del sistema (221 KB)
— .plans/	# Planes de desarrollo (33 KB)
— lydia_agent.egg-info/	# Package metadata
— AGENTS.md	# Guía de desarrollo (1,356 líneas)
— CONTRIBUTING.md	# Guía de contribución
— pyproject.toml	# Dependencias exactas (391 líneas)
— README.md	# Documentación principal (263 líneas)
— LICENSE	# MIT

4.2 Configuración del Sistema (config.yaml)

`config.yaml` es el archivo de configuración principal. Sigue la regla: **.env solo para secrets** (API keys, tokens) y **config.yaml para toda la configuración de comportamiento** (timeouts, thresholds, feature flags, display prefs).

Sección	Propósito	Ejemplos
servers	Configuración de proveedores de modelos LLM	base_url, api_key (en .env), model name, temperature, max_tokens
gateway	Opciones de gateway de mensajería	telegram_token, discord_bot_token, webhook_url
tools	Configuración de herramientas habilitadas	terminal.enabled, browser.enabled, file.enabled, delegation.enabled
memory	Opciones de memoria persistente	auto_save_interval, fuzzy_threshold, fts5_enable
tools	Herramientas habilitadas y configuradas	{enable, disable, enable_all, disable_all}
models	Catálogo de modelos disponibles	{list, switch, set_cost}
skills	Gestión de habilidades	{list, create, update, search}
plugins	Instalación y gestión de plugins	{list, install, enable, disable}
cron	Jobs programados	{list, create, delete, run, update}

Nota: Todos los valores de configuración conductual van en `config.yaml`, nunca en `.env`. Solo las credenciales (API keys, tokens, passwords) se almacenan en `.env`.

4.3 Variables de Entorno

Variable	Propósito	Valores típicos
<code>LYDIA_HOME</code>	Directorio raíz de datos del agente	<code>~/.lydia</code> (nuevo) o <code>~/.lydia</code> (legacy)
<code>LYDIA_MODEL</code>	Modelo predeterminado	<code>custom/qwen3.6-a3b-claude-coder</code>
<code>LYDIA_PROVIDER</code>	Proveedor de modelo	<code>custom</code> (proveedores personalizados)
<code>PYTHONUTF8</code>	UTF-8 encoding en Windows	<code>1</code> (habilitado)
<code>PYTHONIOENCODING</code>	Codificación de entrada/salida	<code>utf-8</code>
<code>TELEGRAM_BOT_TOKEN</code>	Token de bot de Telegram	<code>123456:ABC ...</code>

4.4 Perfiles de Usuario

Los perfiles permiten entornos separados con configuraciones de agente individuales, herramientas y plugins. Cada perfil tiene su propio `skills/` , `plugins/` , `cron/` y `memories/` y opera sin influencia entre perfiles.

Comando	Propósito
<code>lydia profile create</code>	Crear nuevo perfil
<code>lydia profile switch <name></code>	Cambiar a perfil existente
<code>lydia profile list</code>	Listar todos los perfiles
<code>--profile <name></code>	Especificar perfil al iniciar

5. Sistema de Herramientas y Skills

Lydia Agent V2 cuenta con **89 herramientas** (model tools) en el core y **57 skills de dominio** en `optional-skills/`. Las herramientas son la interfaz que el modelo usa para interactuar con el mundo real; las skills son procedimientos reutilizables que el agente carga según la tarea.

5.1 Herramientas del Core (tools/)

Herramienta	LOC	Función
mcp_tool.py	4,911	Gateway MCP: conexión con servidores MCP (Model Context Protocol)
browser_tool.py	4,499	Control de navegador (CamoFox): navegar, click, type, screenshot
skills_hub.py	4,069	Hub de skills: catálogo, carga, búsqueda, ejecución de skills
delegate_tool.py	3,510	Subagentes: delega tareas a agentes hijo en contexto aislado
terminal_tool.py	2,978	Terminal remoto con PTY, background processes, watch patterns
tts_tool.py	2,846	Texto a voz multi-proveedor (Edge, OpenAI, ElevenLabs)
file_operations.py	2,423	Operaciones de archivos: read, write, search, patch
approval.py	2,252	Sistema de aprobación: confirmación antes de acciones destructivas
process_registry.py	2,219	Registro de procesos: gestión de PIDs en background
file_tools.py	1,936	Herramientas de archivo: find, grep, diff, patch
code_execution_tool.py	1,868	Ejecución de código Python sandboxeado con timeout
vision_tools.py	1,735	Análisis de imágenes vía modelos visión
send_message_tool.py	1,794	Envío de mensajes multi-plataforma
transcription_tools.py	1,799	Transcripción de audio a texto
... más 75 herramientas adicionales (77,703 LOC total en tools/)		

5.2 Skills de Dominio (optional-skills/)

Categoría	Skills	Propósito
software-development	14	TDD, debugging, code review, MCP integration, subagent-driven dev
research	22	ArXiv, blog monitoring, wiki knowledge base, market data

creative	7	Arte ASCII, diagramas, diseño web, p5.js, manim, pixel art
devops	4	Kubernetes, Docker, SSH, Gitea, webhook subscriptions
web-development	7	Astro dashboards, Flutter, Go microservices, Node debug
mlops	1	Evaluación LLM, W&B, HF Hub, vLLM, llama.cpp
blockchain	3	Smart contracts, DeFi, análisis on-chain
productivity	4	Notion, Airtable, Google Workspace, PDF editing
email	2	Himalaya CLI: IMAP/SMTP desde terminal
security	5	Análisis de vulnerabilidades, hardening
autonomous-ai-agents	multi	Claude Code, Codex, OpenCode, Hermes deployment
health	2	Seguimiento de salud, recordatorios
finance	3	Mercados financieros, BYMA, ROFEX, MAE
payments	6	Procesamiento de pagos
mcp	4	MCP server management, n8n, mcporter
gaming	multi	Minecraft servers, Pokémon emulator

Nota: Las skills se cargan bajo demanda según el contexto de la conversación. El sistema de skills hub (4,069 líneas) gestiona el catálogo, la sincronización y la ejecución de skills.

5.3 Footprint Ladder (Jerarquía de Implementación)

Para evitar inflar el core, Lydia sigue esta jerarquía al añadir nueva capacidad:

1. **Extender código existente** — la opción más barata
2. **CLI command + skill** — nueva funcionalidad empaquetada como skill
3. **Service-gated tool** (check_fn) — habilitada por chequeo dinámico
4. **Plugin** — extiende el core sin tocarlo
5. **MCP server** — servidor externo via Model Context Protocol
6. **Core tool nueva** — último recurso, requiere justificación

6. Memoria y Aprendizaje Continuo

Lydia es el único agente con un **bucle de aprendizaje incorporado**: crea skills desde la experiencia, las mejora durante el uso, persiste conocimiento entre sesiones, busca sus propias conversaciones pasadas y construye un modelo profundo del usuario.

6.1 Subsistemas de Memoria

Sistema	Tecnología	Propósito
Memory (persistente)	SQLite FTS5	Hechos duraderos: preferencias del usuario, datos de entorno, convenciones del proyecto. Se inyecta en cada turno de conversación. Compacta y prioriza automáticamente.
Session Search	SQLite FTS5	Búsqueda de conversaciones pasadas: recupera sesiones por contenido, scroll dentro de una sesión, resolución de referencias @session:/.
Skills (memoria procedural)	SKILL.md + YAML frontmatter	Procedimientos reutilizables: workflows, comandos, pitfalls. Se crean desde la experiencia y se actualizan automáticamente.
User Profile	Perfil estructurado	Quién es el usuario: nombre, rol, preferencias, estilo de comunicación. Se construye y refina con cada interacción.
Context Engine	Plugin + agent context	Preparación de contexto enriquecido antes de cada turno: memoria + skills + proyectos + últimas sesiones + cron jobs.

7. Cron Jobs y Tareas Programadas

El sistema de cron (7 módulos, ~485 KB) permite programar tareas que el agente ejecuta automáticamente en segundo plano. Las tareas corren en sesiones aisladas y entregan resultados al usuario sin intervención manual.

7.1 Componentes del Sistema Cron

Módulo	Función
cron/scheduler.py	Planificador principal: evalúa schedules, gestiona ejecuciones
cron/jobs.py	Gestión de jobs: crear, listar, actualizar, pausar, eliminar
cron/blueprint_catalog.py	Catálogo de blueprints predefinidos para jobs comunes
cron/suggestion_catalog.py	Sugerencias inteligentes basadas en el uso del usuario
cron/suggestions.py	Generador de sugerencias para automatización

7.2 Capacidades del Cron

- **Schedules flexibles:** cada 30m, cada 2h, cron expression, ISO timestamp
- **Script mode:** jobs que ejecutan scripts directamente sin LLM (watchdog pattern)
- **Agent mode:** jobs que ejecutan prompts con el agente completo
- **Chained jobs:** un job puede consumir el output de otro (context_from)
- **Multi-entrega:** Telegram, Discord, Slack, local, o todas las plataformas
- **Auto-contexto:** los jobs heredan AGENTS.md, .cursorrules del workdir

8. Ecosistema de Plugins

Lydia Agent V2 cuenta con **175 plugins** (~10.8 MB) organizados en categorías funcionales. Los plugins son la vía principal para extender el agente sin modificar el core.

Categoría	Archivos	Items	Propósito
model-providers	28	141	Proveedores de modelo: OpenAI, Anthropic, Google, Ollama, OpenRouter, vLLM, etc.
platforms	51	164	Plataformas de mensajería: Telegram, Discord, Slack, WhatsApp, Signal, etc.
memory	20	46	Backends de memoria: SQLite, MemGPT, knowledge graphs
web	17	59	Herramientas web: scraping, búsqueda, feeds
browser	6	21	Control de navegador: CamoFox, Chromium
image_gen	6	30	Generación de imágenes: ComfyUI, Stable Diffusion
video_gen	2	10	Generación de video
spotify	3	8	Control de Spotify: play, search, queue, playlists
google_meet	14	19	Google Meet: transcripción, resumen, pipeline de reuniones
teams_pipeline	8	9	Microsoft Teams: pipeline de reuniones
kanban	1	multi	Sistema Kanban para gestión de tareas
observability	2	8	Observabilidad y monitoreo
dashboard_auth	4	20	Autenticación para web dashboard
security-guidance	2	6	Guías de seguridad

Los plugins se instalan vía `lydia plugin install <name>` y se gestionan con `lydia plugin list/enable/disable`. Cada plugin vive en su propio directorio bajo `plugins/` y se comunica con el core a través de ABCs y hooks definidos.

9. Seguridad y Control de Acceso

9.1 Capas de Seguridad

Capa	Mecanismo	Descripción
Autenticación	agent/auth.py (80 líneas)	Valida identidad del usuario. Soporta OAuth, tokens API, y autenticación local.
Permisos	agent/permissions.py (169 líneas)	RBAC: roles y permisos granulares para cada herramienta y acción.
Aprobación de Edits	agent/edit_approval.py (287 líneas)	Requiere confirmación humana antes de modificar archivos. Safety crítico.
Procedencia	agent/provenance.py (128 líneas)	Chain of custody: cada acción tiene un audit trail completo.
Path Security	tools/path_security.py	Validación de rutas: impide acceso a directorios fuera del sandbox.
Security Scan	Tools runtime	Escaneo de comandos en terminal: detecta URLs raw, pipes a intérpretes, acceso a redes privadas.

9.2 Gestión de Secrets

`.env` es el único lugar para credenciales (API keys, tokens, passwords). Toda configuración de comportamiento va en `config.yaml`. Esta separación evita que secrets se filtren en commits o logs.

10. Despliegue e Instalación

10.1 Instalación Estándar

```
# Clonar repositorio
git clone https://github.com/Stuko0/lydia-agent.git
cd lydia-agent

# Crear y activar virtualenv
python3 -m venv venv
source venv/bin/activate

# Instalar dependencias exactas
uv sync

# Configurar
cp .env.example .env # Editar con API keys
cp config.example.yaml config.yaml

# Inicializar
lydia setup

# Ejecutar
lydia
```

10.2 Modos de Ejecución

Modo	Comando	Uso
CLI	lydia / lydia run	Modo interactivo en terminal
Gateway daemon	lydia gateway start	Daemon 24/7 para Telegram, Discord, Slack, etc.
TUI	lydia tui	Interfaz de terminal con paneles
Desktop	lydia-desktop	Aplicación Electron con UI gráfica
Web Dashboard	lydia web-server	Dashboard web para gestión
MCP Server	lydia mcp-serve	Sirve herramientas vía MCP

10.3 Proveedores de Modelo

Lydia soporta múltiples backends de inferencia a través de 28 plugins de model-providers:

- **OpenAI** — GPT-4o, GPT-4.1, o1, o3
- **Anthropic** — Claude Opus 4, Sonnet 4, Haiku 3.5
- **Google** — Gemini 2.5 Pro, Flash
- **Ollama (local)** — Qwen3.6, Gemma4, Laguna-XS, DeepSeek, llama.cpp GGUF
- **OpenRouter** — 200+ modelos agregados
- **vLLM** — Serving local de modelos open-weight
- **Custom providers** — Cualquier endpoint compatible con OpenAI API

11. Resumen y Estado del Proyecto

11.1 Métricas del Proyecto

Métrica	Valor
Versión	0.17.0
Líneas de código Python (total)	867,344 LOC
Módulos del core agente (agent/)	106 archivos
Módulos CLI (lydia_cli/)	195 archivos
Herramientas (tools/)	89 (77,703 LOC)
Plugins	175 (~10.8 MB)
Skills de dominio	57 (~5.9 MB)
Gateway platforms	15+
Idiomas (locales)	16
Tests	1,864 archivos (~25 MB)
Runner principal (run_agent.py)	5,709 líneas
Gateway daemon (gateway/run.py)	19,114 líneas
CLI principal (cli.py)	17,116 líneas

11.2 Estado de Componentes

Componente	Estado	Notas
Agent Core	✅ Producción	Maduro, loop de conversación estable
CLI	✅ Producción	Funcionalidad completa
Gateway	🔄 En desarrollo	Daemon funcional, añadiendo plataformas
Cron	🔄 En desarrollo	Funcionalidad base completa, blueprints en expansión
Tools	🔄 En desarrollo	89 herramientas, añadiendo más
Plugins	🔄 En desarrollo	175 plugins, ecosistema creciendo
Skills	🔄 Parcial	57 skills, algunas en desarrollo
Browser (ACP)	🔄 En desarrollo	11 módulos, control remoto funcional

Desktop App	 En desarrollo	Electron wrapper, funcionalidad básica
Tests	 Producción	1,864 tests, cobertura amplia
Documentación	 En desarrollo	SDD, README multi-idioma, AGENTS.md

11.3 Arquitectura de Modelos (Local)

Lydia se conecta a un servidor **Ollama remoto** con RTX A6000 (48GB VRAM) para inferencia local. Los modelos disponibles son:

Modelo	Tamaño	Params	Uso
qwen3.6:35b	23.9 GB	36.0B Q4_K_M	Agente principal: tools + thinking
gemma4:26b	18.0 GB	25.8B Q4_K_M	Alternativa agente general
laguna-xs-2.1	35.6 GB	33.4B Q8_0	Coding agéntico pesado
north-mini-code-1.0	32.4 GB	30.5B Q8_0	Coding agéntico alternativo

El servidor está configurado con OLLAMA_NUM_PARALLEL=4 y OLLAMA_MAX_LOADED_MODELS=3 para permitir inferencia paralela.